

COM2002-1

Profesor: Joaquín Soto

TAREA 1

Estudiante: Consuelo Rodríguez Quezada

Contextualización del problema. El procesamiento de texto es una tarea habitual en los sistemas operativos, ya que permite transformar información y buscar contenido dentro de los datos recibidos. En Linux existen herramientas como `tr` y `grep` que facilitan estas operaciones desde la línea de comandos.

Esta tarea aborda la implementación en C de versiones simplificadas de ambas herramientas. El primer problema consiste en reemplazar todas las ocurrencias de un carácter por otro, utilizando dos argumentos de línea de comandos. El segundo consiste en contar las líneas que contienen al menos una ocurrencia de una palabra indicada como único argumento, contando cada línea una sola vez aunque presente varias coincidencias.

Ambos programas deben procesar texto desde la entrada estándar y entregar sus resultados mediante la salida estándar. El objetivo es aplicar el manejo de caracteres y cadenas, el uso de argumentos y el procesamiento secuencial de la entrada para resolver estas operaciones de forma eficiente.

Pregunta 1

Lógica del código. Para este problema, el código desarrollado recorre la palabra entregada y busca carácter por carácter uno que coincida con el objetivo (el que buscamos reemplazar); cuando encuentra un carácter igual al objetivo, entrega en esa posición al carácter de reemplazo (el que colocamos en lugar del objetivo).

Al comienzo, la función toma como argumento `argc` y `argv`, como el número de argumentos y el arreglo con los argumentos respectivamente. Luego, tenemos dos `If`, que buscar parchar los casos en que:

- se entreguen más o menos de 2 argumentos,
- se entreguen strings o caracteres especiales.

Se definen como `char` el carácter objetivo igual al primer argumento, y el carácter de reemplazo igual al segundo argumento. Se decidió también colocar un `printf` con la instrucción de introducir la palabra.

Previo al ciclo `While`, se define como `char` la variable de `actual`. Al definir el `While`, leemos `actual` con `getchar()`, que lee el siguiente carácter disponible, y mientras `actual` no sea salto de línea: iteramos.

Si `actual` es igual al carácter objetivo, con `putchar()` colocamos el carácter de reemplazo en su lugar, y si `actual` no es el carácter objetivo, lo colocamos en su lugar tal cual le corresponde.

¿Qué entradas utilizaría para evaluar el correcto funcionamiento de su programa? Probaría primero ingresando más y menos de dos argumentos, números, strings y caracteres especiales.

¿Cómo reacciona su programa si se le entregan más de dos argumentos o si los argumentos ingresados contienen más de un carácter? Si se le entregan más de dos argumentos, el programa arroja un error con el mensaje *"Se admiten dos argumentos, ni más ni menos"*.

Si se le ingresan argumentos con más de un carácter, va a lanzar un error con un mensaje que dice *"Debe ingresar caracteres unitarios, no palabras o frases"*

¿Cómo modificaría su programa para que permita eliminar un carácter en lugar de reemplazarlo? Yo modificaría el interior del `While`, luego del `If` que reconoce si `actual` es igual a `objetivo`, en lugar de colocar `putchar()` saltaría con un `continue` para que no coloque el carácter objetivo.

Casos de prueba y resultados. Se presentan en las imágenes algunas de las pruebas realizadas al código. El proceso consistió de probar más y menos de dos argumentos, para lo cual se había parchado en el código. También se probó con números y caracteres especiales, como letras con acentos, los cuales si reconoció. En cuando a la palabra, se logró ingresar además frases

```

● (base) consu@Poquito:~/PSS/Tarea_1$ gcc Pregunta_1.c -o Pregunta_1
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 o i
Ahora, introduzca una palabra o frase: orocoipo
● iriciipi(base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 3 a
Ahora, introduzca una palabra o frase: 330923
● aa092a(base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 3 5
Ahora, introduzca una palabra o frase: tengo 23 vacas
○ tengo 25 vacas(base) consu@Poquito:~/PSS/Tarea_1$
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 A o
Ahora, introduzca una palabra o frase: hoIA
● holo(base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 a A

```

```

● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 A o
Ahora, introduzca una palabra o frase: hoIA
● holo(base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 a A
Ahora, introduzca una palabra o frase: aparca
○ ApArcA(base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 aaaaa a
Debe ingresar caracteres unitarios, no palabras o frases
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 a oisd
Debe ingresar caracteres unitarios, no palabras o frases
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 a hola
Debe ingresar caracteres unitarios, no palabras o frases
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1 efe
Se admiten dos argumentos, ni más ni menos
● (base) consu@Poquito:~/PSS/Tarea_1$ ./Pregunta_1
Se admiten dos argumentos, ni más ni menos

```

Pregunta 2

Lógica del código. Para esta pregunta, el código consiste de dos funciones, la función `compare_words()` que realiza la comparación carácter a carácter de dos palabras; y de la función `main()` que realiza el resto de las operaciones.

La función `compare_words()` admite como argumentos los punteros hacia dos palabras, y mientras estas sean iguales, va a ir recorriéndolas. Retorna 1 si ambas son completamente iguales, y 0 si no lo son.

En `main()`, se reciben como argumentos `argc` y `argv`, la cantidad de argumentos recibidos en la línea de comando, y el arreglo con esos argumentos respectivamente. Primero, revisamos que se haya recibido únicamente una palabra como argumento. Luego definimos `palabra`, como el argumento recibido, `counter` como un contador para las líneas, y `linea` para leer fila a fila el archivo.

Abrimos un ciclo `While` que va a leer línea a línea (hasta que alguna sea nula), y dentro primero definimos `word`, que será la variable para leer cada palabra. En un ciclo `For` recorreremos la línea, carácter a carácter, dentro vamos a definir `largo` (largo de `word`) como 0, ya que inicialmente `word` está vacío, y dentro, en un ciclo `While` vamos a leer palabra a palabra.

Como funciona el ciclo `While` más interno, es que vamos a entrar a él siempre y cuando el carácter que leemos no sea una especie de separador; al entrar, vamos a ver si el largo de `word` es menor a su tamaño menos el último carácter (no sobrepasa su tamaño), y si cumple, colocamos el carácter al final de la palabra y aumentamos el largo de `word`. Cuando este `While` se encuentra con un separador (espacio, coma, punto, etc), se sale de este bucle y asignamos como último carácter de `word` al nulo.

Luego, si `palabra` y `word` son iguales, aumentamos el contador y le hacemos `break` al for más interno para saltar el resto de la línea. Finalmente, retornamos `word` a ser un char vacío, y si no hemos llegado al final de la línea, aumentamos `i`. Por último, printeamos `counter` y retornamos 0 si todo se ejecutó correctamente.

¿Qué sucede si una misma línea contiene la palabra buscada más de una vez? ¿Cómo asegura su programa que esa línea solo se cuente una única vez? En una línea que contenga la misma palabra dos veces, el código debiera leer solamente la primera y luego saltar de línea, por lo que cuenta sólo una vez esa línea.

¿Cómo modificaría su programa para que la búsqueda sea insensible a mayúsculas y minúsculas (case-insensitive)? Podría utilizar la función `strcascmp` de la librería `<strings.h>`, que compara cadenas completas sin importar mayúsculas, en lugar de la función `compare_words`.

¿Cómo afectaría el rendimiento de su programa si la palabra a buscar es muy larga o si el archivo procesado tiene millones de líneas? Si la palabra buscada es demasiado larga, comparar dos palabras puede requerir revisar más caracteres, especialmente si coinciden al principio. Sin embargo, la función termina en cuanto ambas son iguales, o cuando encuentre una diferencia, por lo que no va a recorrer siempre toda la palabra. Además, al crear `word` se fija un límite de caracteres.

Si el archivo tiene millones de líneas, aumentará el tiempo de ejecución porque el programa debe leer y procesar todo el texto. El trabajo depende principalmente de la cantidad total de caracteres, no sólo de el número de líneas. El `break` ayuda por que deja de buscar una línea cuando encuentra la primera coincidencia.

Por último, el consumo de memoria se mantendría constante, ya que se procesa una línea a la vez y se reutilizan los arreglos sin almacenar el archivo completo. Eso sí, también se fija un límite para los caracteres admitidos en una línea, si este límite se sobrepasa, `fgets()` la dividiría y podría contarla como dos o más líneas diferentes.

Casos de prueba y resultados. Los casos de prueba consistieron en probar argumentos dentro de lo aceptado por el código, palabras unitarias, no frases, y se logró que compliara correctamente. También se probó ingresarle frases, para lo cual el código tiró el error de necesitar sólo un argumento. También se lograron leer números y caracteres especiales sin problema.